Expert Systems With Applications 124 (2019) 209–225

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

A variable neighborhood search algorithm for the bin packing problem with compatible categories

Luiz F.O. Moura Santos[a], Renan Sallai Iwayama[b], Luísa Brandão Cavalcanti[c], Leandro Maciel Turi[b], Fabio Emanuel de Souza Morais[d], Gabriel Mormilho[d], Claudio B. Cunha[b,d,*]

[a] Department of Production Engineering, University of São Paulo, São Paulo, Brazil
[b] Graduate Program in Logistics Systems Engineering, University of São Paulo, São Paulo, Brazil
[c] Department of Naval Architecture and Ocean Engineering, University of São Paulo, São Paulo, Brazil
[d] Department of Transportation Engineering, University of São Paulo, São Paulo, Brazil

ARTICLE INFO

Article history:
Received 8 November 2018
Revised 16 January 2019
Accepted 23 January 2019
Available online 24 January 2019

Keywords:
Bin packing problem with compatible categories
Last mile distribution
Nanostores
Variable neighborhood search

ABSTRACT

In this paper, we address the Bin Packing Problem with Compatible Categories (BPCC), a challenging optimization problem that arises in the context of last mile distribution to nanostores in large cities, particularly in developing countries. By introducing the concept of incompatible categories of items to be delivered (i.e., types of items that cannot be transported together, such as food and cleaning products), as opposed to the item-by-item incompatibility found in previous literature, we aim to determine the best assignment of deliveries of distinct products to a homogeneous fleet of capacitated vehicles in order to minimize the number of required vehicles (bins). To solve large instances of the BPCC that are commonly found in practice, we propose an efficient variable neighborhood search (VNS) metaheuristic that relies on a simple greedy heuristic to generate initial solutions and on simple and efficient neighborhoods as well as problem-tailored shaking procedures. We perform extensive computational experiments on a very large set of 8000 instances derived from benchmark datasets. The tested instances differ in terms of number of items (ranging from 201 to 1002), bin capacities, compatibility matrices and proportion of items that belong to the different categories. We also present two new mathematical formulations for the BPCC and compare them to the item-by-item bin packing formulation (Gendreau et al., 2004) using a high performance computer (HPC). The computational experiments indicate that our VNS algorithm can effectively solve the BPCC in very short CPU times.

© 2019 Elsevier Ltd. All rights reserved.

1. Introduction

As the urban population of developing countries continues to rise at high rates, last-mile delivery remains a key challenge to retailers, who attempt to be as close to consumers as possible. In such countries, a significant share of the retail market corresponds to very small stores – henceforth addressed as nanostores – located in densely populated areas, in which customers can find nearly everything they need, and also benefit from informal credit (Fransoo, Blanco, & Mejia-Argueta, 2017). Since nanostores are very limited in space, shippers face the challenge to ensure on-shelf availability of their products, usually making frequent visits to these locations in order to deliver small amounts of a limited range of Stock Keeping Units (SKUs) (Gámez Albán, Soto Cardona, Mejía Argueta, & Sarmiento, 2015). To accomplish that, delivery salespeople are usually employed to actively supply nanostores, driving small vehicles that are more adequate to reach these areas, packed with a large variety of SKUs from different suppliers. These professionals also monitor inventories, determine replenishment needs, perform the sales transactions, collect payment and deliver items in one single visit (Fransoo et al., 2017; Gámez Albán et al., 2015). From a transportation planning perspective, this means that vehicles must be loaded with small quantities of cargo that varies in size, value and demand to supply as many products as possible in a single visit.

In this context, we address the problem of last-mile delivery of items to nanostores, which is modeled as the Bin Packing with Compatible Categories (BPCC). The BPCC is a variant of

* Corresponding author at: Department of Transportation Engineering, University of São Paulo, São Paulo, Brazil.
E-mail addresses: luiz.fel.moura@usp.br (L.F.O.M. Santos), renan.siw@usp.br (R.S. Iwayama), luisa.cavalcanti@usp.br (L.B. Cavalcanti), leandro.turi@usp.br (L.M. Turi), fabio.emanuel@gmail.com (F.E.d.S. Morais), gmormilho@gmail.com (G. Mormilho), cbcunha@usp.br (C.B. Cunha).

https://doi.org/10.1016/j.eswa.2019.01.052
0957-4174/© 2019 Elsevier Ltd. All rights reserved.

the widely studied Bin Packing Problem (BPP), which belongs to the Knapsack Problems class and has been proven to be NP-Hard (Martello & Toth, 1990a). Our goal is to assign deliveries of distinct products to a homogeneous fleet of capacitated small vehicles, assuring that all demand is met and only items from compatible (i.e., non-conflicting) categories are packed together, in a way that minimizes the number of required vehicles (bins). While item categories refer to different product types, such as cleaning and hygiene products, food, beverage, etc., a conflict between a pair of categories denotes some risk that may occur (such as cross-contamination or damage) if they are loaded together in the same vehicle. Although such incompatibilities also apply to the traditional retail industry, in the nanostore context they must be explicitly modeled because, since the amount of each SKU transported is small, the variety of products that can fit a single vehicle is greater than usual (Gámez Albán et al., 2015). Moreover, as nanostores are usually located in distant, poor suburbs of big cities and are adjacent to each other, the distances of line-haul legs of the delivery routes (i.e., between the depot and the delivery area) are significantly greater than those within the delivery area. Therefore, the fixed costs of the delivery vehicles (i.e., capital, labor, license and insurance costs) prevail over the variable per-mile costs (fuel and maintenance) and largely overcome any distance-related cost saving that could be achieved at the expense of an additional vehicle (or trip) that might be required. Thus, the main concern in such context is to minimize the total number of vehicles (bins) required to attend all customers.

Despite the literature on the BPP being extensive (see, e.g., Coffman, Csirik, Galambos, Martello, & Vigo, 2013; Delorme, Iori, & Martello, 2016), works focused on bin packing with conflicts are still scarce. Few researchers have approached the Bin Packing Problem with Conflicts (BPC), another variant of the BPP introduced by Jansen and Öhring (1997) and further explored by Gendreau, Laporte, and Semet (2004) and Muritiba, Iori, Malaguti, and Toth (2010). However, while in the BPC conflicts arise between individual items in a pairwise manner, the BPCC introduces a new practical concept of compatible categories of items, in which each item of a given category conflicts with all items that belong to a given incompatible category.

According to Hemmelmayr, Schmid, and Blum (2012), bin packing problems belong to a class of well-studied and highly popular combinatorial optimization problems. In line with Muritiba et al. (2010) for the BPC, the BPCC can also be considered NP-hard, once it is a special case of the BPC as conflicts arise between all pairs of items that belong to incompatible categories, and it reduces to the BPP if compatibility constraints are dropped. Thus, to solve it efficiently, we propose a Variable Neighborhood Search (VNS) framework, and test it against four different sets of instances derived from difficult *augmented* Non-IRUP and *augmented* IRUP BPP instances proposed by Delorme et al. (2016), totaling 8000 instances. These sets differ in terms of bin capacities (the original and three additional capacities are evaluated), compatibility matrices (six and eight categories) and the proportion of items among the different categories. We also present two new mathematical formulations to help evaluating the quality of our solutions, and we compare them to the original formulation that consider disjunction constraints (Gendreau et al., 2004).

Since, to our knowledge, this is the first appearance of the BPC in which conflicts arise not between pairs of individual items but among categories of products, our paper makes the following contributions: it (1) introduces the BPCC and a new mathematical modeling; (2) presents an efficient VNS algorithm to obtain good results in short computational times, applicable to the context of delivery planning for physical distribution of products to nanostores; and (3) proposes a new comprehensive set of benchmark instances for the BPCC, so that future work results can be compared to the VNS algorithm proposed here.

The remaining of this article is organized as follows: a literature review on the BPP, its variations and solution methods are presented in Section 2. The new mathematical formulations for the BPCC are given in Section 3, while our VNS algorithm for solving the problem is described in Section 4. Section 5 contains the computational experiments, results, and discussion. Finally, concluding remarks and future research directions are presented in Section 6.

2. Literature review

In the literature, the problem faced by traditional retail suppliers and shippers has been modeled as a combination of packing (or loading) and routing problems, like the 3-dimensional Loading Capacitated Vehicle Routing Problem (3L-CVRP) introduced by Gendreau, Iori, Laporte, and Martello (2006) and further explored by other authors (Bortfeldt & Homberger, 2013; Fuellerer, Doerner, Hartl, & Iori, 2010; Zhu, Qin, Lim, & Wang, 2012). In a similar way to our nanostore last-mile distribution problem, a solution to the loading part of the 3L-CVRP must specify a set of goods to be carried by each vehicle, ensuring that the goods' total weight does not exceed the vehicle weight capacity (Gendreau et al., 2006) – which also matches the Bin Packing Problem (BPP) formulation (Dowsland & Dowsland, 1992). However, neither the 3L-CVRP and nor the BPP consider incompatibilities between items, i.e., both assume that no item has any conflict with others, and thus can be packed together.

Respen and Zufferey (2017) addressed a real-world truck loading problem from a car manufacturer in which parts need to be loaded in trucks in order to be delivered to manufacturing plants. As each truck can deliver goods to many delivery points, classes of items are defined with the aim to ensure that all items belonging to a class (which is associated to a single plant) are unloaded, since rearranging items in the truck at a stop is not allowed. Although the problem seems to be related to the BPP, it was modeled as a special type of rectangular strip packing problem (SPP). As with the SPP, in which rectangular items must be packed in a single bin of fixed width and infinite length, trucks are loaded sequentially, based on the resulting optimized sequence. No compatibility constraints between individual items or classes are present. The authors also point out that an extension of the problem would consider all trucks to be jointly (instead of sequentially) loaded.

Conflicts between individual pairs of objects have been introduced to the BPP by Jansen and Öhring (1997); however, the problem is modeled as a scheduling problem in which objects are represented by tasks and machines represent bins. Different approximation algorithms for solving the BPC are presented and evaluated. They rely on a conflict graph $G=(J, E)$, where J denote all jobs (items) to be scheduled (packed) and E is the set edges connecting jobs that cannot be processed by the same machine. Such representation of the problem has also been adopted by other authors (i.e., Gendreau et al., 2004; Jansen, 1999; Khanafer, Clautiaux, & Talbi, 2010; Muritiba et al., 2010; Sadykov & Vanderbeck, 2013), who explicitly address the problem of Bin Packing with Conflicts.

Jansen (1999) focuses on approximation algorithms, while Gendreau et al. (2004) present new upper and lower bounds for the BPC, establishing its first benchmark. Their upper bounds are obtained by applying six different heuristics to the problem, combining greedy procedures with vertex coloring methods. Such benchmark was later updated by Muritiba et al. (2010), who employed an exact approach, based on a set covering formulation solved with a Branch-and-Price algorithm.

Khanafer et al. (2010) propose techniques that improve some of Muritiba et al. (2010) lower bounds, based on data-dependent dual-feasible functions (DDFF) and its generalized version (GDDFF).

The authors also present new lower bounds for the Two-Dimensional Bin Packing Problem with Conflicts (BPC-2D). Similarly to Muritiba et al. (2010), Sadykov and Vanderbeck (2013) also applied a generic Branch-and-Price algorithm for solving the BPC, but with a dynamic programming algorithm for pricing when the conflict graph is an interval graph, while for the general case a depth-first-search branch-and-bound approach is used.

However, as mentioned before, all cited works consider compatibility among items in a pairwise manner, instead of defining compatibility across categories (each category comprising a set of items), which has nudged researchers to develop graph-oriented solution methods.

Even though the heuristics commonly applied to solve combinatorial optimization problems have not been tested on the BPC, they have been proposed for the BPP, as greedy heuristics do not provide solutions that are sufficiently accurate. For instance, applications of the Variable Neighborhood Search (VNS) are reported by Fleszar and Hindi (2002) for bins of different sizes and by Hemmelmayr et al. (2012) as well. Stawowy (2008) apply an evolutionary-based algorithm, which is similar to an Iterated Local Search approach. Masson et al. (2013) address of the problem of allocating a set of processes across a pool of server machines as a multi-capacity bin packing problem, a generalization of the classical bin-packing problem in which the machine (bin) capacity and tasks (items) sizes are given by multiple dimensions; the authors show that their iterated local search heuristic produces high-quality solutions in a reasonable computational time, even for large-scale instances. Pereira and Vilà (2015) present a formulation based on the one-dimensional minimax bin-packing problem with bin size constraints for the test assembly design problem that consists in selecting the optimal subset of items (questions) among those available to define one or several tests (questionnaires) subject to specific constraints and objectives. A Grouping Genetic Algorithm with Controlled Gene Transmission (GGA-CGT) is proposed by Quiroz-Castellanos et al. (2015). More recently, Buljubašić and Vasquez (2016) propose a consistent neighborhood search approach for solving the BPP, based on Tabu Search (TS) and hill climbing/descent procedures. The authors also present a generalization of the method for the Two-Dimensional Vector Packing Problem (2-DVPP), tested on instances from the literature.

Solution methods for the BPP usually start with an initial solution obtained by a greedy simpler heuristic, namely the First-Fit (FF) and the Best-Fit (BF) algorithms, which are renamed as First-Fit Decreasing (FFD) and Best-Fit Decreasing (BFD) if objects are pre-sorted in non-increasing weight order before being assigned to bins (Coffman et al., 2013). Worst-case performance of such algorithms have been mathematically demonstrated by Delorme et al. (2016), showing its results are not consistently good. Furthermore, tests made by Stawowy (2008) show that there is no significant difference between using the solution provided by FF, FFD, BF or BFD algorithms as the initial solution to a metaheuristic. On the other hand, Quiroz-Castellanos et al. (2015) show slight modifications on such methods that may improve the results.

All above-mentioned greedy initial heuristics have been adapted for the BPC. For instance, Gendreau et al. (2004) adopt the First-Fit Decreasing with Conflicts to produce an initial solution, which consists in performing a compatibility check at each step of the algorithm, when an item is to be placed in a bin. Such check is also used by Muritiba et al. (2010), who apply for the Best-Fit Decreasing with Conflicts. They also evaluated a new approach, named Worst Fit Decreasing with Conflicts, in which each item is assigned to the bin with largest available capacity (instead of the smallest, as in the BFD) among those whose compatibility is not violated.

3. The bin packing problem with compatible categories

In this section, we propose a mathematical formulation for the BPCC that explicitly takes into account conflicts among items that belong to incompatible categories. It derives from formulations for the BPC and the BPP proposed by Gendreau et al. (2004) and Delorme et al. (2016), respectively.

We are given a set $C=\{1, 2, \ldots, p\}$ of categories; a set $J=\{1, 2, 3, \ldots, n\}$ of items, each item $j \in J$ weighting $w_j \in \mathbb{Z}^+$and belonging to a category $cat_j \in C$; an infinite number of identical bins of capacity $b \in \mathbb{Z}^+$; and a compatibility matrix C_{pxp} populated with binary elements c^{kl} that equals 1 if category $k \in C$ is compatible with category $l \in C$ and 0 otherwise. Items belonging to categories that are in conflict cannot be assigned to the same bin (i.e., items j and r can't be packed together if $c^{cat_j cat_r}=0$). We also define p subsets $J^k \subset J$, comprising all items $j \in J$ such that $cat_j = k$, $\forall j \in J^k$ and assume $b \geq w_j \; \forall j \in J$.

The objective of the BPCC is to assign all items to the minimum number of bins while ensuring that the sum of the weights of all items assigned to a bin does not exceed its capacity b and that all items assigned to the same bin belong to compatible (i.e., non-conflicting) categories. The problem can be modelled by introducing three sets of binary decision variables:

(i) y_i that equals 1 if bin i is used and 0 otherwise (as in the worst case the number of bins equals n, $i=1, 2, 3, \ldots, n$);
(ii) x_{ij} that equals 1 if item j is assigned to bin i and 0 otherwise ($i=1, 2, 3, \ldots, n$; $j=1, 2, 3, \ldots, n$);
(iii) f_i^k that equals 1 if at least one item belonging to category k is assigned to bin i and 0 otherwise ($i=1, 2, 3, \ldots, n$; $k=1, 2, 3, \ldots, p$).

Then, the BPCC can be expressed as follows:

$$\min z = \sum_{i=1}^{n} y_i \tag{1}$$

Subject to:

$$\sum_{j \in J} x_{ij} * w_j \leq b y_i \qquad i = 1, 2, \ldots, n \tag{2}$$

$$\sum_{i=1}^{n} x_{ij} = 1 \qquad j \in J \tag{3}$$

$$\sum_{j \in J^k} x_{ij} \leq M f_i^k \qquad \begin{array}{c} i = 1, 2, \ldots, n \\ k \in C \end{array} \tag{4}$$

$$f_i^k + f_i^l \leq 1 \qquad \begin{array}{c} i = 1, 2, \ldots, n \\ \forall k, l \in C : c^{kl} = 0 \end{array} \tag{5}$$

$$y_i \geq y_s \qquad i, s \in \{1, 2, \ldots, n\} : i < s \tag{6}$$

$$y_i, x_{ij}, f_i^k \in \{0, 1\} \qquad \begin{array}{c} \forall i, j \in J \\ \forall k \in C \end{array} \tag{7}$$

The objective function (1) minimizes the total number of required bins. Constraints (2) are the classical capacity constraints of BPP that ensure bins' capacities are respected whereas they force variables y_i to equal 1 when at least one item has been assigned to bin i. Constraints (3) impose that each item is assigned to exactly one bin. In constraints (4), M denotes a sufficiently large number (e.g., the number of items n) to link decision variables x_{ij} and f_i^k in such a way that if an item belonging to category k is assigned to a bin i, then f_i^k equals 1. Constraints (5) enforce that for every pair of incompatible categories, at most one of them is present in the same bin, for every used bin. Although constraints (6) are

not required, they reduce the problem's solution space by ensuring unused bins have higher indexes than used ones, so if, for instance, 4 bins are needed to pack an instance with 50 items, they ought to be the bins identified by 1, 2 3 and 4 (as in y_1, y_2, y_3 and y_4), discarding the remaining 46 bins. Therefore, despite not being mandatory for the formulation to properly represent the BPCC, constraints (6) aim to speed up the optimization.

It should be noted that constraints (4) could alternatively be replaced by (8). We evaluate both alternatives in the computational experiments' session.

$$x_{ij} \leq f_i^k \qquad \begin{array}{l} i \in \{1, 2, \ldots, n\} \\ k \in C, \;\; j \in J^k \end{array} \tag{8}$$

4. VNS metaheuristic

To solve large instances of the BPCC that are commonly found in practice, we propose a variable neighborhood search (VNS) algorithm. Introduced by Mladenović and Hansen (1997), VNS is a metaheuristic that uses systematic changes in the neighborhood structure of a search. Starting from a given initial solution x_0, the solution space is explored by means of a finite and predetermined set of neighborhood structures $\mathcal{N}_h = (h_1, h_2, \ldots, h_{\max})$ that allows jumping from increasingly distant neighborhoods of the current solution to a new one, in order to escape from local optimality (Hansen & Mladenović, 2001; Hemmelmayr et al., 2012). For a recent survey on the VNS literature, please refer to Hansen, Mladenovic, Brimberg, and Moreno Pérez (2010).

In the following subsections, we describe how we generate an initial solution x_0 and the predetermined set of four neighborhood structures $\mathcal{N}_h = (h_1, h_2, h_3, h_4)$, where $\mathcal{N}_h(x)$ denotes the set of solutions in the hth neighborhood of the incumbent solution x.

4.1. Construction heuristic

To obtain feasible initial solutions, our VNS uses a greedy procedure based on the First-Fit Decreasing (FFD) algorithm, hereafter referred to as First-Fit Category Decreasing (FFCD). Recall that the term 'decreasing' means items are pre-sorted with the intent of first allocating items that are larger and, supposedly, more difficult to fit into the existing bins. Such strategy is adapted to the BPCC by assuming that it is better to pack less-flexible items first, since it becomes harder to assign them to bins as the number of already packed items increases. In other words, we first attempt to allocate items that belong to categories that are incompatible with a greater number of categories. Hence, instead of sorting items based on their weight and then allocating them one-by-one in the first possible bin (as in the FFD), our approach initially sorts elements based solely on their category, using a measurement that reflects how (in)compatible a category is; then, within each category, its elements are sorted according to their respective weights.

To evaluate the (in)compatibility level of a category, we simply check the amount of the remaining categories that are compatible with it. Expression (9) presents a proper definition of such index, hereafter referred to as the degree of compatibility of category k, denoted by dc^k.

$$dc^k = \sum_{l=1, l \neq k}^{p} c^{kl} \qquad \forall\, k \in C \tag{9}$$

After sorting items non-decreasingly by dc^{cat_j}, for each category $k \in C$ FFCD sorts all corresponding items $j \in J^k$ in a non-increasing order of their weights w_j. At each stage of the algorithm, all items $j \in J^k$ (i.e., of a single category k) are packed into the bins using the FFD procedure (Coffman et al., 2013) as follows: the first unassigned item j of the previously ordered set J^k is packed into the first feasible bin, i.e., a bin with enough residual capacity whose items are compatible with the new item j being added. The FFCD heuristic is summarized in Algorithm 1.

Algorithm 1 FFCD heuristic for the BPCC.

Start: FFCD for the BPCC.
Step 1: *Sort categories $k \in C$ in non – decreasing order of dc^k;*
For (*each category $k \in C$*) **do**:
Step 2: *Sort items $j \in J^k$ in non – increasing order of weight;*
Step 3: *Assign all items $j \in J^k$ using FFD;*
Step 4: **Next** *category.*
End: FFCD for the BPCC.

Algorithm 2 VNS for the BPCC.

Start: VNS for the BPCC.
1: $x \leftarrow x_0$;
2: $x_{best} \leftarrow x$;
3: $s_1 \leftarrow 1$; $s_2 \leftarrow 1$; $s_3 \leftarrow 1$; $s_4 \leftarrow 1$;
4: **While** (stopping criteria not met) **do**
5: $h \leftarrow$ *ChooseNeighbourhood*(s_1, s_2, s_3, s_4);
6: $x' \leftarrow$ *Shaking*(x, h);
7: $x'' \leftarrow$ *LocalSearches*(x');
8: **If** $(g(x'') > g(x_{best})$ and $z(x'') \leq z(x_{best}))$ **then**
9: $x_{best} \leftarrow x''$;
10: $s_h \leftarrow s_h + 1$;
11: **End – if**;
12: $x \leftarrow x_{best}$;
13: **End – while**;
End: VNS for the BPCC.

4.2. Variable neighborhood search

According to Fleszar and Hindi (2002), a crucial factor to any neighborhood search procedure is the choice of the objective function. The authors point out that, for the BPP, using the real objective function of the problem (i.e., minimizing the number of bins) can be rather pointless, since items can be arranged in many different ways, but still corresponding to the same number of bins. As better solutions tend to have fuller bins and inspired by Quiroz-Castellanos et al. (2015), we adopt an additional occupation function $g(x)$ (henceforth referred as fitness function) with the aim of obtaining solutions with lower residual capacities. Our proposed fitness function is given by (10), where N_i denotes the set of items $j \in J$ that are assigned to bin i (i.e., such that $x_{ij} = 1$), and n_{bins} is the number of used bins of a given solution x.

$$g(x) = \sum_{i=1}^{n_{bins}} \left(\sum_{j \in N_i} w_j x_{ij} \right)^2 \tag{10}$$

One feature of this function is that it favors solutions that have some nearly full bins and some nearly empty bins rather than one with several equally filled bins. In addition to $g(x)$, the original objective function (1) is also considered when updating the best solution x_{best}, which means that only solutions with the same or fewer total number of bins are considered.

As in Hemmelmayr et al. (2012), instead of using all the neighborhoods equally, we make use of an adaptive function named *ChooseNeighbourhood*(s_1, s_2, s_3, s_4). It selects the neighborhood structure to be used in the current iteration according to a probability that is determined based on each score s_1, s_2, s_3 and s_4, given by expression (11). These scores are computed based on the number of successful iterations obtained by each shaking operator.

$$P(\mathcal{N}_h) = \frac{s_h}{\sum_{i=1}^{h_{max}} s_i} \tag{11}$$

The pseudocode of our VNS algorithm is presented in Algorithm 2. Procedures *Shaking*(x, h) and *LocalSearches*(x') (lines 6 and 7, respectively) are detailed in the following subsections.

4.2.1. Shaking

In the VNS literature, shaking operators are employed to randomly obtain new solutions in the neighborhood of the current incumbent solution. Hence, the function *Shaking*(x, h) of our algorithm randomly generates a x' solution in the hth neighborhood of x, i.e. $\mathcal{N}_h(x)$, using the following set of neighborhood operators:

- $\mathcal{N}_1$ randomly chooses a category $k \in C$ for which all items are removed from their currently assigned bins. These items are then repacked using the FFCD heuristic;
- $\mathcal{N}_2$ is similar to $\mathcal{N}_1$, but randomly chooses two (instead of one) categories to be removed from all bins, and then repacked by applying FFCD;
- $\mathcal{N}_3$ makes use of two random factors. Firstly, a number of bins to be fully emptied is randomly determined, respecting a parameter α that sets the maximum percentage of bins that can be emptied in a given solution. The bins to be emptied are then randomly chosen among all existing bins; lastly, FFCD is employed to repack the removed items;
- $\mathcal{N}_4$ is an outer loop of $\mathcal{N}_3$ and works in the same way, but instead explore farther away neighborhoods by emptying a larger number of bins than $\mathcal{N}_3$. It is also driven by a parameter β that sets the maximum percentage of bins that can be erased within the solution, but in a way such that $\beta > \alpha$.

Whenever the chosen operator yields to an improved solution, its respective score is incremented. In other words, the operators in our algorithm are selected adaptively reflecting their individual success during the search.

These shaking operators are inspired on previous works of Gendreau et al. (2004) and Hemmelmayr et al. (2012) for related bin packing problems. Differently from the latter, our VNS has focused solely on "ruin-and-recreate" (R&R) operators, given the nature of our problem. The R&R principle was proposed by Schrimpf, Schneider, Stamm-Wilbrandt, and Dueck (2000) for solving traveling salesman problems, vehicle routing problems and network optimization problems. It relies on the fact that by ruining a fraction of the solution and then restoring it can yield to improved solutions, particularly in the case of discontinuous problems with many constraints, as is the case of the BPCC.

Hence, since in the BPCC bins are all equal in terms of their capacities, improvements in the objective function (i.e., reducing the total number of used bins) can be more effectively achieved if we are able to improve the packing process by changing compatibility relationships and arrangements of items in some of the bins, as the local search operators are constrained by the capacity and compatibility constraints of the problem.

These operators are also similar to the adaptive large neighborhood search (ALNS) framework presented in Ropke and Pisinger (2006), an extension of the large neighborhood search proposed by Shaw (1998) in which a number of simple algorithms compete to modify the current solution. In each iteration, an algorithm is chosen to destroy the current solution, and an algorithm is chosen to repair the solution. The idea of adaptive search is that the parameters of the algorithm are no longer fine-tuned but adjusted dynamically using the information obtained during the search. Thus, a roulette wheel selection based on previously obtained scores is used for choosing a destroy and a repair neighborhood; these scores are automatically adjusted during the search favoring the ones with a higher success rate in the previous iterations. However, differently from the ALNS algorithm, we do not accept deteriorating solutions with some probability, as in a simulated annealing. In addition, we use only one construction heuristic (FFCD) to rebuild a full solution given a partial solution that resulted from applying a destroy-type shaking procedure.

Finally, our approach also presents similarities to other VNS approaches from the literature that were proposed to related problems. For instance, Dusberger and Raidl (2015) employed some very large neighborhood structures based on the R&R principle to solve the 3-staged 2-dimensional cutting stock problem. Pereira et al. (2015) addressed a special case of the BPP by a VNS in which similar shaking procedures, comprising the removal of elements from the solution and their reintroduction using a constructive greedy heuristic, are applied.

4.2.2. Local search

Two different local search procedures are performed within every iteration, following a shaking performed by neighborhood $\mathcal{N}_h$.

The first local search procedure $\mathcal{L}_1$ aims to increase the average density of the bins, by moving items from bins with lower used capacities to others more occupied. Procedure $\mathcal{L}_1$ takes the incumbent solution and runs through all used bins. For every identified bin i_A, each of its items is checked if it fits into any other bin i_B with higher used capacity (i.e., $\sum_{j \in N_{i_B}} w_j x_{ij} > \sum_{j \in N_{i_A}} w_j x_{ij}$). It should be noted, though, that moves are performed only if: (i) the residual capacity of bin i_B after the move is smaller than the residual capacity of bin i_A before the move; (ii) it does not lead to a violation in terms of the bin i_B capacity nor yield to a category conflict. $\mathcal{L}_1$ ends when there are no more items that can be moved.

The second local search $\mathcal{L}_2$ is based on swaps between two items that are assigned to different bins in the current solution provided by $\mathcal{L}_1$. It uses the concept of ranking all feasible swaps based on their improvements to the fitness function (expression 10). The ranking comprises only swaps that improve the fitness function and that do not involve any bins that already took part of previous swaps in the current iteration. Due to the high computational cost of recalculating the fitness function for each swap, this is done incrementally, as described by Fleszar and Hindi (2002). In other words, it is not necessary to have the fitness function completely recalculated every time, but just the changes due to each swap. As mentioned before, only swaps that lead to improvements in the fitness function are considered. Procedure $\mathcal{L}_2$ ends after all possible swaps have been made. It is worth noting that local search procedures $\mathcal{L}_1$ and $\mathcal{L}_2$ are performed in sequence.

The VNS procedure terminates whenever one of three stopping criteria is met: a number λ of iterations without improvements is reached; a total of ϕ iterations is reached; or an optimum solution is found (i.e., if its objective function equals the obvious lower bound that can be computed in $O(n)$ by expression 12, as suggested by Martello & Toth, 1990b).

$$L_{CONT} = \left\lceil \left(\frac{\sum_{i=1}^{n} w_i}{b} \right) \right\rceil \tag{12}$$

5. Computational experiments

5.1. Proposed benchmark instances

The BPCC instances used in our experiments are derived from the *augmented non*-IRUP (ANI) and the *augmented* IRUP (AI) sets of difficult benchmark instances proposed by Delorme et al. (2016). IRUP stands for the Integer Round-Up Property, which indicates that it is possible to calculate the optimal value of the problem by solving its linear relaxation and adopting the smallest integer equal to or greater than the value thus obtained. Such value is commonly adopted as a lower bound (L_{CONT}) for BPPs (Caprara, Dell'Amico, Díaz-Díaz, Iori, & Rizzi, 2015; Gendreau et al., 2004; Khanafer et al., 2010; Muritiba et al., 2010). According to Delorme et al. (2016) and Caprara et al. (2015), it is more difficult to prove optimality for the instance set ANI that do not respect this property, since objective function values of the optimal solutions are higher than the respective L_{CONT}. Original instances comprise five sets of 50

Table 1
Compatibility matrix for six categories.

	C_1	C_2	C_3	C_4	C_5	C_6	Typical products
C_1	1	0	1	0	0	0	Hygiene, cleaning and others
C_2	0	1	0	1	1	1	Sweets, pasta, salt
C_3	1	0	1	1	0	0	Tobacco
C_4	0	1	1	1	1	1	Soft and alcoholic drinks
C_5	0	1	0	1	1	0	Eggs
C_6	0	1	0	1	0	1	Chips, cans, dressings, oil, cookies

ANI and 50 AI instances, totaling 500 instances. Each instance belonging to the same ANI set has the same number of items $n \in \{201, 402, 600, 801, 1002\}$, while the AI sets have $n+1$ items. The original instances are available at http://or.dei.unibo.it/library/bpplib; more details on how they were generated can be found in Delorme et al. (2016).

In the original BPP instances, the sum of the weights of all items results in an integer value that is a multiple of the capacity of the bins (Delorme et al., 2016). As our problem addresses a BPP related to last mile distribution to nanostores, it is rather uncommon that, in practice, the total amount of products to be delivered yields to vehicles being loaded to their full capacity. In an attempt to make instances more realistic, we considered four distinct capacity factors: 100%, 120%, 150% and 200% of their respective original capacities. Thus, our main computational experiments comprise 2000 tested instances with respect solely to the number of the items to be packed and the capacity of the bins. Regarding the latter, it should also be noted that, in original instances, the largest item corresponds to about 90% of the respective bin capacity, and consecutive objects (when items are sorted non-increasingly by their weights) are also large-sized items as compared to the bin capacity. Therefore, increasing original bins capacities also make the instance set more diverse and realistic in terms of the items sizes in comparison with the respective bin capacity, once it is very unlikely, in practice, that the amount of products of a single category to be delivered to a specific nanostore fills the majority of a vehicle's freight compartment.

The last modification of previous instances introduces the BPCC categories to the datasets, a crucial point of the BPCC. For our purpose, each item must be assigned to one of six distinct categories as observed in real-world problems related to delivering to nanostores. The corresponding pairwise category compatibility matrix is depicted in Table 1, where one denotes two compatible categories and zero otherwise.

Such compatibility constraints can also be represented by a compatibility graph $G=(V, A)$, in a way that is similar to the approach used for the BPC. In our case, the nodes of the graph denote the different categories $k \in C$ of items and an arc $(k, l) \in A$ is created for each compatible pair of categories $k, l \in C$, i.e., $c^{kl}=1$. In this representation, the degree of compatibility of a category k is equal to the number of arcs that are connected to node k. Fig. 1 shows a representation of graph G, in which the size of each nodes is directly proportional to the degree of compatibility of its corresponding category. Note that category 1 is the least compatible one, category 4 is the most compatible one, and categories 3, 5 and 6 are equivalent in terms of degree of compatibility.

To assign categories to the items of the original instances, we propose a simple rule to determine the category of the first item of each instance. To determine the first category, we consider the remainder of the division by 6 of the last one or two numerical digits that identify the specific instance (for example, instance 0 starts with an item that belongs to category 6, and the following

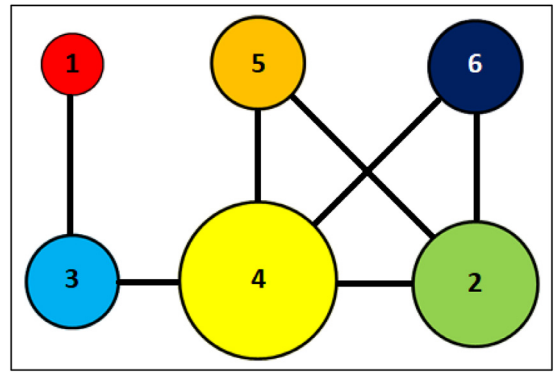

Fig. 1. BPCC compatibility graph for instances with six categories: vertices sizes are proportional to their respective degrees of category compatibility.

items are designated to categories 1, 2, 3, 4, 5 and 6; similarly, the first item of instance 1 is assigned to category 1; the first of instance 2 is designated to category 2, of instance 6 to category 6, of instance 7 to category 1 again and so on, given that instances are numbered from 0 to 49). Obviously, if the remainder is zero, (e.g., instances whose final digits are 6, 12, 18 and so on), the first item is assigned to category 6.

Despite the large size of the above instance set (500 problem instances and four different bin capacities, totaling 2000 instances), we also generated two additional subsets of 2000 instances that differ in the sense that items are not evenly distributed over all the six categories.

In this case, based on the category that is assigned to the first item as described above, the assignment of the subsequent items to categories is made in such a way that it results in some categories with more items than others. More precisely, for each instance, two new types of unbalanced number of categories were generated: with two and four large categories. In the first case, the two first categories of the instance have twice as many items as the other four categories, while in the second case the four initial categories (instead of two) have twice as many items as the two remaining ones. To exemplify this procedure, let us consider an instance whose last identification digit is 3; in this case, we assign category 3 to the first item, followed by the next item being assigned to category 4, and then the items are assigned to categories 5 and 6. Prior to returning to category 1, the two subsequent items are assigned to categories 3 and 4, so that the number of items belonging to these two categories are doubled. Then, the next items are assigned according to the ascending order of categories (1, 2, 3, 4, 5, 6), followed by categories 3 and 4 again. This procedure is repeated until all items have been designated a category. An equivalent procedure is applied in the case of four large categories, which can also be seen as having only two small categories.

We also generated an additional subset of instances with eight categories that also mimic what arises in practice. The eight categories are balanced; in other words, each category has, on average, the same number of objects. Categories are assigned to items in the same way as for the subset that corresponds to the six categories in which items are evenly distributed and are given for all four capacity factors, totaling other 2000 instances. The corresponding pairwise category compatibility matrix and its network representation are depicted in Table 2 and Fig. 2, respectively; the same convention (regarding node size) also applies. Item sizes and bin capacities remain the same as in the previous subsets.

The resulting test bed is composed by 8000 instances in total.

Table 2
Compatibility matrix for eight categories.

	C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8	Typical products
C_1	1	1	0	0	0	0	0	0	Cleaning
C_2	1	1	0	0	1	0	0	0	Hygiene
C_3	0	0	1	1	1	1	1	0	Canned food
C_4	0	0	1	1	0	1	1	1	General food (Non-canned)
C_5	0	1	1	0	1	1	0	0	Tobacco
C_6	0	0	1	1	1	1	1	0	Soft drinks & water
C_7	0	0	1	1	0	1	1	0	Alcohol
C_8	0	0	0	1	0	0	0	1	Fresh groceries

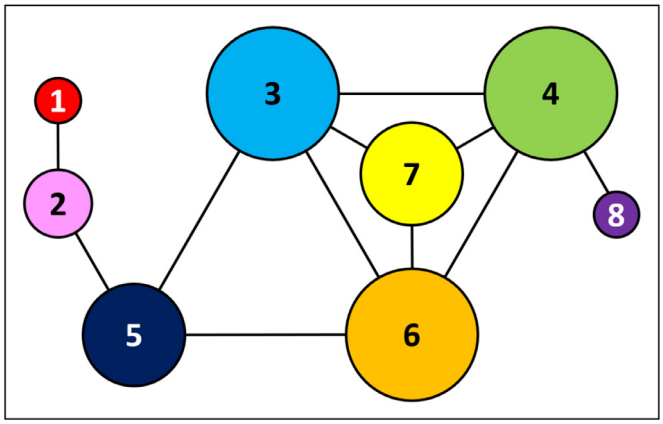

Fig. 2. BPCC compatibility graph for instances with eight categories: vertices sizes are proportional to their respective degrees of category compatibility.

5.2. Comparison of alternative formulations and lower bounds

In our initial experiments, we used the commercial optimization solver Gurobi 7.0 running on a standard desktop personal computer (PC) and C++ environment to exactly solve the problem. The PC has an Intel® Core™ i7-4770 CPU @ 3.4 GHz with 16 GB of RAM. However, as the optimizer failed to reach optimality even for most of the smaller instances (recall that the problem is NP-hard (Coffman et al., 2013; Gendreau et al., 2004; Martello & Toth, 1990a)), we opted for using the high-performance computing (HPC) resources of University of São Paulo's Advanced Scientific Computing Laboratory (LCCA). More specifically, in our experiments, we used the cluster named 'Aguia', with 64 physical servers (nodes); each server has an Intel® Xeon® CPU E7-2870 @ 2.40 GHz with 10 cores, 20 threads and 512 GB of RAM.

The HPC cluster uses the Portable Batch System (PBS) (Henderson, 1995) as a queuing system to manage jobs, and runs the commercial solver IBM ILOG CPLEX version 12.6 in parallel processing mode, using up to 20 threads per node (though in our experiments, jobs were submitted to a maximum of one node at a time). Utilizing a massive paralell processing power was the only way to determine the optimal solutions for some of the instances of this hard combinatorial problem. Submitted jobs contained several instance problems; each problem to be executed within a time limit of 5 h. The Integer Linear Programming (ILP) model for each instance fed to CPLEX in the HPC was generated in a *.lp* input file, running Gurobi in the PC.

Since larger instances took several hours solely to be loaded into CPLEX, and for many of them it was not even possible to find feasible integer solutions within the time limit, instances were split into two sets: small and large instances. Small instances are those with 200 and 400 items (i.e., $n \in \{201, 402\}$ for the AI set and $n \in \{202, 403\}$ for the ANI set, respectively), while the large ones have 600 items or more. Given the huge effort (i.e., processing time) required, we considered only the evenly distributed 6-category subset of small instances for the comprehensive tests that involved determining the optimal solutions. For the three remaining subsets of small instances (two unevenly distributed 6-category subsets and one evenly distributed 8-category subset), we selected only 10 instances (20%) from each set of 50 ANI and 50 AI instances, and all capacity factors (totaling 480 instances), due the enormous additional processing time required to have them processed in the HPC.

Our experiments also comprise evaluating the alternative formulations for the BPCC presented in Section 3, namely *CatBigM* (expressions 1–7) and *Cat* (expressions 1–3;5–8), as well as the original BPC formulation (Gendreau et al., 2004; Muritiba et al., 2010) based on items' pairwise conflicts (named *Items*). To determine which formulation should be used in the exact experiments, we submitted the evenly distributed 6-category set that corresponds to 120% capacity factor for the small set of instances (i.e., 200 and 400 items, broadly speaking) to the HPC, and evaluated their performances according to the results we obtained. In order to reduce computational times, an upper bound given by the FFCD solution was also used to limit the number of available bins in the objective functions of all three formulations. To better assess the performance of the three alternative formulations, we also kept record of building times for each submitted instance problem. The results are shown in Table 3.

It is possible to observe that formulation *Items* takes very long times solely to be build and solved, once it requires adding a new

Table 3
Results comparing three different models.

Instance type[a]	Cap. factor (%)	# Items	Model	Gurobi build times[b] (PC)		CPLEX run times[c] (HPC)		
				Mean time (s)	Std. Dev. time (s)	# Opt.	Mean time (s)	Std. Dev. time (s)
AI	120	202	*Cat*	0.734	0.018	49	363.833	1097.665
			CatBigM	0.376	0.009	39	111.684	626.403
			Items	19.927	0.396	44	2951.685	2891.038
		403	*Cat*	3.240	0.252	20	834.096	1349.540
			CatBigM	1.577	0.111	18	419.244	803.312
			Items	171.884	19.614	7	7148.651	1121.287
ANI	120	201	*Cat*	0.732	0.030	47	355.340	1471.419
			CatBigM	0.378	0.017	39	357.048	1857.405
			Items	19.890	0.565	41	2235.545	2838.167
		402	*Cat*	3.416	0.181	20	949.514	2124.473
			CatBigM	1.670	0.114	16	382.866	833.570
			Items	190.340	10.370	7	5527.933	1112.247

[a] Evenly distributed 6-category subset.
[b] Mean and standard deviation build times are given for all instances.
[c] Mean and standard deviation run times for CPLEX are given for instances where optimum reached.

Table 4
Lower bounds results.

Set of instances[a]	Martello and Toth LBs[b]			
	L_1	L_2	L_3	L_{CPLEX} *
Small	59,374	59,640	59,648	59,587
Large	238,776	242,728	242,738	–

[a] Evenly distributed 6-category subset.
[b] Values represent the sum of lower bounds for all problem instances.

constraint for every pair of incompatible items. The *CatBigM* formulation is the fastest but fails to reach optimality within the given time limit for several instances. On other hand, the *Cat* formulation yields to the best results in terms of the greatest number of optimal solutions with reasonable runtimes. Based on this, we chose the *Cat* formulation for our experiments involving the subset of small instances for which the optimal solutions could be determined.

We have also performed an evaluation of three simple lower bounds (LBs) originally proposed by Martello and Toth (1990b) for the BPP:

- $L_1 = L_{CONT}$, the obvious lower bound given by expression (12);
- L_2, indicated for problems with large items when compared to the bin capacity;
- L_3, determined using a reduction procedure.

We refer the interested reader to Martello and Toth (1990b) for more details on how the lower bounds are calculated.

The results obtained with these three lower bounds for the same set of small instances are summarized in Table 4 and compared to the best integer lower bounds provided by CPLEX (L_{CPLEX}). These results indicate that L_3 yields to the best lower bound for the BPCC. In fact, we decided to compute the three lower bounds and use the best guess to evaluate the performance of our proposed VNS algorithm in terms of optimality, which led us to confirm that L_3 is the best LB among L_1, L_2 and $L_3$for all instance problems. When compared to L_{CPLEX}, L_3 yielded the best LB value in approximately 97.63% of the instance problems (out of 800 from the small set), while L_{CPLEX} achieved the best LB value in 93.25% of these cases. However, as previously mentioned, L_1 was the only LB used as one of the stopping criteria in our VNS, due to its simplicity to be computed as an input to the heuristic.

5.3. Tuning

We performed extensive experiments to tune and determine the best setting of parameters α and β for our VNS algorithm. To this end, we carried out an experiment involving several different combinations of these two parameters that were run considering 36% of all balanced instances that corresponds to evenly distributed 6-category subset. The results we have obtained are presented in Table 5, in which bold values denote the best result for each column.

These results show that our VNS is quite insensitive to the values of α (which sets the maximum percentage of bins that can be emptied in a given solution) and β (which sets a higher maximum percentage). Though the differences in the number of used bins (column *"Obj. Sum"*) appear to be very low, the superior performance of the selected parameter setting ($\alpha = 0.05$, $\beta = 0.075$) is evidenced by the fact that it reached the highest numbers of both optimal and best-known solutions, while still presenting very high fitness values. Hereinafter, for our remaining experiments, we chose the aforementioned values, although other settings might yield to similar results.

Table 5
Tuning results.

α	β	# Opt.[a]	# BKS[a]	Obj. sum[b] (%)	Fitness sum[b] (%)
0.05	1.2 α	**225**	**720**	**0.0000**	**+0.0348**
	1.5 α	**225**	**720**	**0.0000**	+0.0340
	2 α	**225**	719	+0.0009	+0.0335
	2.5 α	**225**	719	+0.0009	+0.0329
0.1	1.2 α	**225**	**720**	**0.0000**	+0.0256
	1.5 α	224	716	+0.0037	+0.0220
	2 α	224	719	+0.0009	+0.0240
	2.5 α	**225**	**720**	**0.0000**	+0.0237
0.15	1.2 α	**225**	719	+0.0009	+0.0191
	1.5 α	**225**	719	+0.0009	+0.0177
	2 α	224	718	+0.0018	+0.0174
	2.5 α	224	718	+0.0018	+0.0162
0.2	1.2 α	**225**	**720**	**0.0000**	+0.0126
	1.5 α	224	719	+0.0009	+0.0106
	2 α	**225**	718	+0.0018	+0.0100
	2.5 α	224	719	+0.0009	+0.0097
0.25	1.2 α	224	718	+0.0018	+0.0056
	1.5 α	223	718	+0.0018	+0.0064
	2 α	**225**	719	+0.0009	+0.0058
	2.5 α	224	719	+0.0009	+0.0050
0.3	1.2 α	224	718	+0.0018	+0.0053
	1.5 α	224	717	+0.0027	+0.0025
	2 α	224	718	+0.0018	+0.0010
	2.5 α	224	718	+0.0018	0.0000

[a] Out of 720 instances of the evenly distributed 6-category subset.
[b] Relative to the minimum value from the parameter setting.

Table 6
Used parameters and stopping criteria.

Procedure	λ	ϕ	α	β
VNS (Algorithm 2)	$\lfloor n/2 \rfloor$	$5n$	0.05	$0.075 = 1.5\,\alpha$

Tuning results also suggest that using small values for α and β tend to lead to better solutions. As shaking procedures are based on the concept of destruction and reconstruction of solutions, strong shakings can lead solutions to neighborhoods that are very far from the global optimum, where the local search procedures are caught in local optima. In other words, such strong shakings can lead to a high number of unsuccessful iterations, thus reaching the stopping criteria faster and achieving poor solutions.

5.4. Results

As the total number of tested instances is huge, results are presented in an aggregated way. To accomplish this, we grouped instances according to their type (using the above-mentioned AI/ANI instance classification), number of items and capacity factors, resulting in subsets of 50 instances each.

The proposed VNS algorithm was coded in C++ using Microsoft Visual Studio 2017 and ran on the above-mentioned PC, using the parameters presented in Table 6. Since our heuristic relies on movements that are randomly selected, we ran ten independent runs with different seeds for each problem. Each seed is an integer number used as input for the random number generator. The stopping criteria were set according to the number of items from each instance. The CPLEX time limit was set to 5 h.

5.4.1. Evenly distributed 6-category subset

For the evenly distributed 6-category subset, the results we have obtained using our VNS algorithm for the small instance subsets, as well as some corresponding optimal solutions that we could determine, are presented in Table 7. It should be noted that not all instances could be solved to optimality and only L_{CPLEX} was used as lower bound during the HPC optimization. Columns *"# Opt."* indicate the number of instances for which optimal solutions

Table 7
Results for the small instances of the evenly distributed 6-category subset.

Instance set			FFCD		CPLEX 12.6 (ran on HPC)					VNS (ran on PC)				
Type	# Items	Cap. Factor (%)	# Opt.[b]	Total # Bins	# Opt.[a]	Avg. Gap (%)[a]	Total # Bins	Mean time[c] (s)	Std. Dev. time[c] (s)	# Opt.[b]	Avg. Gap (%)[b]	Total # Bins (Best)	Mean time[d] (s)	Std. Dev. time[d] (s)
AI	202	100	5	3300	5	1.364	3300	626.898	931.188	5	1.364	3300	0.291	0.129
		120	39	2782	49	0.036	2771	296.954	1047.426	50	0	2771	0.111	0.118
		150	50	2200	50	0	2200	5.059	1.774	50	0	2200	N/A	N/A
		200	50	1650	50	0	1650	6.171	2.845	50	0	1650	N/A	N/A
	403	100	1	6650	1	0.737	6650	650.810	N/A	1	0.737	6650	3.710	1.627
		120	28	5671	20	0.772	5658	834.096	1349.540	40	0.176	5658	2.322	1.612
		150	2	4455	1	1.100	4454	6536.130	N/A	2	1.079	4454	2.760	1.330
		200	1	3350	1	1.463	3350	2252.040	N/A	1	1.463	3350	1.784	0.785
ANI	201	100	2	3300	2	1.455	3300	369.090	2.828	2	1.455	3300	0.284	0.126
		120	37	2782	47	0.107	2771	342.869	1465.554	48	0.071	2771	0.147	0.149
		150	50	2200	50	0	2200	4.837	2.031	50	0	2200	N/A	N/A
		200	50	1650	50	0	1650	6.092	3.102	50	0	1650	N/A	N/A
	402	100	2	6650	2	0.722	6650	1154.320	721.447	2	0.722	6650	3.575	1.654
		120	28	5671	20	0.791	5658	949.514	2124.473	40	0.176	5658	2.218	1.550
		150	2	4455	2	1.079	4454	10120.310	4907.519	2	1.079	4454	2.601	1.236
		200	0	3350	0	1.493	3350	N/A	N/A	0	1.493	3350	1.715	0.763

[a] Best integer lower bounds obtained by CPLEX after a 5 h time limit (L_{CPLEX}) were used.
[b] Either L_3 or L_{CPLEX} lower bounds were used.
[c] Mean and standard deviation times for CPLEX are given only when optimal solutions could be reached.
[d] Mean and standard deviation times for VNS are given for all instances and different seeds.

could be found for both CPLEX and our heuristic. For the evaluation of the heuristic, the best lower bound among L_3 and L_{CPLEX} was considered for each instance problem (as L_3 overcomes L_1 and L_2 in all cases). The columns *"Total # Bins"* reports the sum of objective function values (i.e., the total number of required bins for each heuristic instance in the correspondent class). Mean CPU times and their corresponding standard deviations are also presented for each instance class, considering all ten independent runs. In the case the FFCD heuristic yields proven optimal solutions (it happens when the number of bins equals the lower bound), the VNS is not applied; in this case, corresponding CPU times are labeled as non-available (N/A).

The results in Table 7 show that even for the classes of small instances we could not obtain all the optimum solutions, and the difficulty increases as the number of items grows, as evidenced by CPU times required by CPLEX in HPC. On the other hand, our proposed heuristic could reach optimal solutions for more instances than CPLEX and in much shorter processing times, even using just a regular PC.

It should be noted that some caution should be exercised when attempting a direct comparison of running times between CPLEX and our heuristic, since, as mentioned above, CPLEX takes advantage of the massive computing resources of the HPC.

In order to illustrate how our heuristic effectively allow a significant change in the initial solution obtained by the FFCD construction heuristic, Figs. 3 and 4 show the different solutions for a randomly selected problem instance. More specifically, Fig. 3 shows the solution given by the FFCD procedure, where the different colors denote the different product categories using the same colors as in the compatibility graph (Fig. 1). The final solution obtained by our VNS, with a reduction of two bins, is depicted in Fig. 4. In this specific case, both VNS and CPLEX managed to find the optimum number of bins (113), but CPLEX took a much longer time: 816.6 s against 3.1 s for the VNS, respectively. As mentioned above, although these times cannot be directly compared, they indicate a large discrepancy in running times, of two orders of magnitude, despite CPLEX being run in a high-performance computing environment.

Table 8 presents the results for the remaining larger instances, with 600 items or above, for which optimal solutions (or even any feasible integer solution) could not be determined by the HPC within 2 h.

In Table 9 we present some additional results derived with the aim of analyzing the performance of our VNS with respect to variation of the results due to the random decisions made by the methods and to variations in problem characteristics as well. For this, in columns labeled *"Total # Bins (Best)"* and *"Total # Bins (Worst)"* we report the lowest and the highest total number of required bins obtained for the 50 instances of each type (AI/ANI), number of items and capacity factors. The columns labeled *"Diff."* denote the difference between the sum of the objective functions between the best and worst solutions, while *"% of Bests"* indicates the relative number of times that the best-known solution is found by each heuristic, considering the different seeds. These results show that there is no difference between the best and worst solutions (in terms of total number of bins) for the majority of the instances sets (29 of a total of 40, or 72.5%). Even in the few cases in which such difference arises, the average relative deviation is only 0.072% (2÷2271) in the worst case (instance sets AI-202-120% and ANI-201-120%). The results presented in Table 9 also suggest that it is more difficult to solve instances sets for intermediate capacity factors: i.e., all (except two) instance sets whose capacity factors equal 120% of respective original capacities in the original BPP instances of Delorme et al. (2016), as well as two ANI sets whose capacity factor is 150%.

Table 8
VNS results for the large instances of the evenly distributed 6-category subset.

Instance set			FFCD			VNS				
Type	# Items	Cap. Factor (%)	# Opt.[a]	Avg. Gap (%)[a]	Total # Bins	# Opt.[a]	Avg. Gap (%)[a]	Total # Bins (Best)	Mean time[b] (s)	Std. Dev. time[b] (s)
AI	601	100	0	0.503	9950	0	0.503	9950	18.095	6.100
		120	0	4.409	8643	0	4.298	8633	12.619	5.894
		150	0	0.767	6651	0	0.767	6651	10.034	4.907
		200	0	1.000	5000	0	1.000	5000	8.824	4.291
	802	100	0	0.383	13,301	0	0.376	13,300	42.316	16.002
		120	0	5.013	11,637	0	4.929	11,627	35.246	15.218
		150	13	0.667	8910	39	0.353	8882	7.460	12.612
		200	50	0	6650	50	0	6650	*N/A*	*N/A*
	1003	100	0	0.336	16,656	0	0.300	16,650	99.172	30.060
		120	0	5.955	14,731	0	5.917	14,725	83.644	32.288
		150	18	1.797	11,309	23	1.691	11,297	47.941	27.880
		200	0	0.599	8350	0	0.599	8350	53.989	25.409
ANI	600	100	0	0.503	9950	0	0.503	9950	17.571	6.382
		120	0	4.409	8643	0	4.298	8633	12.439	5.348
		150	0	0.767	6651	0	0.767	6651	9.930	4.588
		200	0	1.000	5000	0	1.000	5000	8.201	4.148
	801	100	0	0.383	13,301	0	0.376	13,300	42.173	16.279
		120	0	5.013	11,637	0	4.929	11,627	34.866	14.581
		150	11	0.690	8912	39	0.353	8882	6.930	12.418
		200	50	0	6650	50	0	6650	*N/A*	*N/A*
	1002	100	0	0.342	16,657	0	0.300	16,650	99.590	29.558
		120	0	5.955	14,731	0	5.917	14,725	86.110	31.259
		150	18	1.806	11,310	23	1.691	11,297	49.439	29.311
		200	0	0.599	8350	0	0.599	8350	53.270	24.000

[a] The L_3 lower bounds (Martello & Toth, 1990b) were used for these instances.
[b] Mean and standard deviation times for VNS are given for all instances and different seeds.

Table 9
Analysis of the performance of the VNS algorithm (evenly distributed 8-category subset).

Instance set			FFCD	VNS			
Type	# Items	Cap. Factor (%)	Total # Bins	Total # Bins (Best)	Total # Bins (Worst)	Diff.	% of Bests
AI	202	100	3300	3300	3300	0	100.0
		120	2782	2771	2773	2	99.6
		150	2200	2200	2200	0	100.0
		200	1650	1650	1650	0	100.0
	403	100	6650	6650	6650	0	100.0
		120	5671	5658	5661	3	98.8
		150	4455	4454	4454	0	100.0
		200	3350	3350	3350	0	100.0
	601	100	9950	9950	9950	0	100.0
		120	8643	8633	8634	1	99.6
		150	6651	6651	6651	0	100.0
		200	5000	5000	5000	0	100.0
	802	100	13,301	13,300	13,300	0	100.0
		120	11,637	11,627	11,627	0	100.0
		150	8910	8882	8882	0	100.0
		200	6650	6650	6650	0	100.0
	1003	100	16,656	16,650	16,650	0	100.0
		120	14,731	14,725	14,727	2	98.4
		150	11,309	11,297	11,299	2	98.8
		200	8350	8350	8350	0	100.0
ANI	201	100	3300	3300	3300	0	100.0
		120	2782	2771	2773	2	99.6
		150	2200	2200	2200	0	100.0
		200	1650	1650	1650	0	100.0
	402	100	6650	6650	6650	0	100.0
		120	5671	5658	5661	3	98.8
		150	4455	4454	4454	0	100.0
		200	3350	3350	3350	0	100.0
	600	100	9950	9950	9950	0	100.0
		120	8643	8633	8635	2	99.2
		150	6651	6651	6651	0	100.0
		200	5000	5000	5000	0	100.0
	801	100	13,301	13,300	13,300	0	100.0
		120	11,637	11,627	11,627	0	100.0
		150	8912	8882	8883	1	99.4
		200	6650	6650	6650	0	100.0
	1002	100	16,657	16,650	16,650	0	100.0
		120	14,731	14,725	14,727	2	98.0
		150	11,310	11,297	11,298	1	99.2
		200	8350	8350	8350	0	100.0

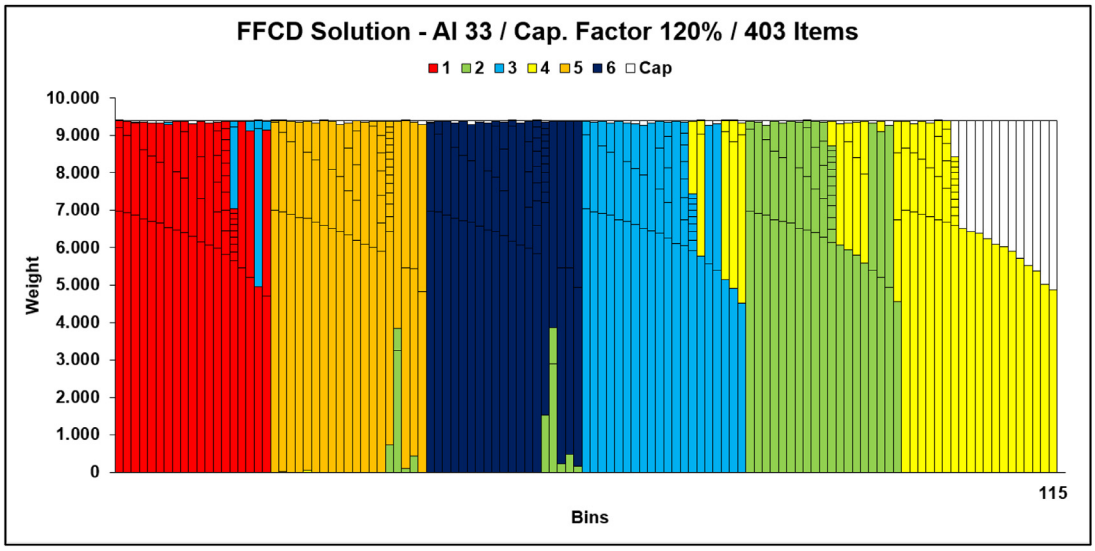

Fig. 3. Illustration of an initial solution generated by the FFCD algorithm for an evenly distributed 6-category instance.

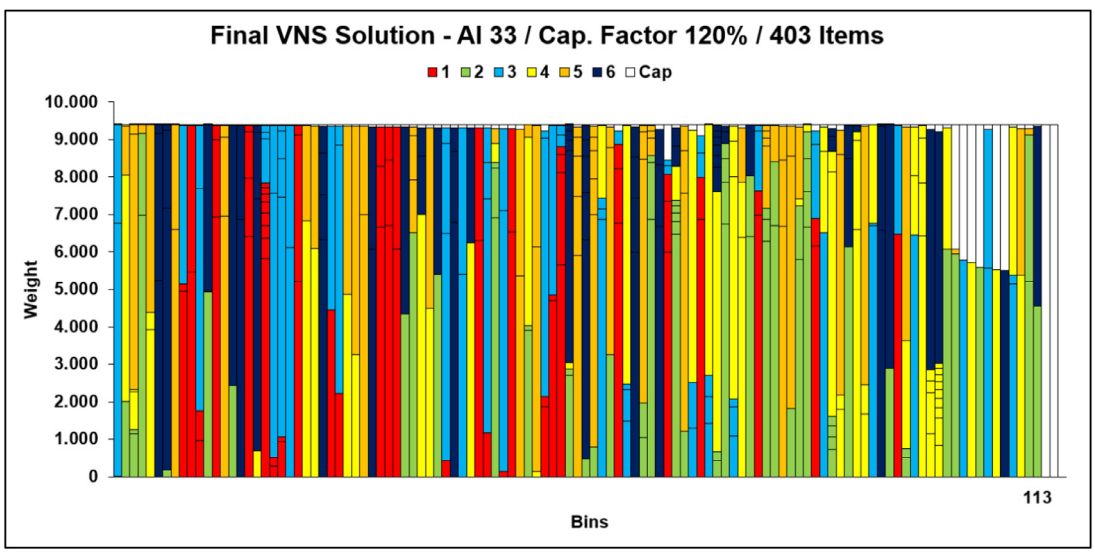

Fig. 4. Illustration of a final optimal solution after the VNS procedure for an evenly distributed 6-category instance.

5.4.2. Evenly distributed 8-category subset

Table 10 presents the CPLEX and VNS results for tested small instances with eight categories (Fig. 2 and Table 2), while Table 9 presents the results of VNS for all tested instances with eight categories. The headers of the tables have the same meanings as Tables 7 and 8, respectively. As with the evenly distributed 6-category subset, not all instances could be solved to optimality and only L_{CPLEX} was used as lower bound during the HPC optimization. Because of the extensive time to run all instances for the 5 h time limit set to CPLEX, each subset depicted in Table 10 comprises only 10 instances, or 20% of the total number of instances (50). The best lower bound among L_3 and L_{CPLEX} was considered for each instance problem solved by VNS. Again, whenever the FFCD heuristic yields proven optimal solutions (i.e., when the number of bins equals the lower bound), the VNS is not executed; in this case, corresponding CPU times are labeled as *N/A*.

Table 10 shows that we were not able to obtain the optimum solutions for some small instances. As with what happened with the former subset of instances, the VNS heuristic could reach optimal solutions for more instances than CPLEX and in much shorter processing times, despite the fact a regular PC was used to test VNS. As mentioned before, a direct comparison of running times between CPLEX and our VNS requires caution, since CPLEX takes advantage of the massive computing resources of the HPC.

In summary, with respect to the quality of the solution, the results are very similar to those for the small instances of the evenly distributed 6-category subset (Table 7) in terms of relative performance of the VNS in comparison with CPLEX. On the other hand, VNS running times on a PC tend to be slightly higher for the 8-category subset, while average CPLEX times are lower in similar proportions for only the few instances for which the optimal solutions could be reached. In other words, finding the optimal solutions remains a challenge as the number of categories increases, since for only a few instances optimal solutions could be found despite the massive processing power provided by the HPC.

Table 10
CPLEX and VNS results for the small instances of the evenly distributed 8-category subset.

8-Category Instance set			FFCD			CPLEX 12.6 (ran on HPC)						VNS (ran on PC)				
Type	# Items	Cap. Fact (%)	# Opt.[b]	Avg. Gap (%)	Total # Bins	# Opt.[a]	Avg. Gap (%)	Total # Bins	Mean time[c] (s)	Std. Dev. time[c] (s)	# Opt.[b]	Avg. Gap (%)	Total # Bins	Mean time (s)	Std. Dev. time (s)
AI	202	100	2	1.212	660	2	1.212	660	6579.210	8572.878	2	1.212	660	0.455	0.241
		120	2	1.419	562	9	0.179	555	1180.632	2153.011	8	0.357	556	0.265	0.171
		150	10	0	440	10	0	440	4.125	1.885	10	0	440	N/A	N/A
		200	10	0	330	10	0	330	6.354	9.436	10	0	330	N/A	N/A
	403	100	0	0.752	1330	0	0.752	1330	N/A	N/A	0	0.752	1330	4.566	2.010
		120	5	0.520	1145	3	0.870	1142	205.953	235.663	8	0.174	1141	3.325	1.727
		150	0	1.118	895	1	1.011	894	2046.230	N/A	1	1.011	894	3.749	1.496
		200	0	1.493	670	0	1.493	670	N/A	N/A	0	1.493	670	2.583	0.839
ANI	201	100	6	0.606	660	6	0.606	660	3313.168	2233.041	6	0.606	660	0.404	0.170
		120	2	1.419	562	9	0.179	555	203.347	265.478	9	0.179	555	0.228	0.205
		150	10	0	440	10	0	440	4.361	1.515	10	0	440	N/A	N/A
		200	10	0	330	10	0	330	3.458	1.872	10	0	330	N/A	N/A
	402	100	0	0.752	1330	0	0.752	1330	N/A	N/A	0	0.752	1330	4.901	2.410
		120	5	0.520	1145	4	0.780	1141	998.875	1491.799	8	0.174	1141	3.319	1.700
		150	0	1.118	895	0	1.119	894	N/A	N/A	1	1.011	894	4.149	1.843
		200	0	1.493	670	0	1.493	670	N/A	N/A	0	1.493	670	2.332	1.031

[a] Best integer lower bounds obtained by CPLEX after a 5 h time limit (L_{CPLEX}) were used.
[b] Either L_3 or L_{CPLEX} lower bounds were used.
[c] Mean and standard deviation times for CPLEX are given only when optimal solutions could be reached.

For the large instances, as with the 6-category subset, VNS results depicted in Table 11 also evidence the effectiveness of our VNS in producing high-quality solutions in similar very low running times.

5.4.3. Unevenly distributed 6-category subset

Table 12 presents the CPLEX and VNS results for the small instances of the two subsets of unevenly distributed items among six categories: U2 (i.e., with two large categories) and U4 (i.e., with four large categories). Although the number of bins required to pack all items is slightly higher when compared to the previous evenly distributed 6- and 8-category subsets, the results are, in general, very similar in terms of running times for the CPLEX and the VNS, and for the quality of the solutions as well.

Tables 13 and 14 show the results of VNS for all U2 and U4 instance subsets, respectively. Again, the results are consistent with the previous ones for evenly distributed 6- and 8-category subsets, evidencing the performance of our VNS.

5.5. Analysis of the neighborhood structures

As we need to keep track of the success rate for each shaking neighborhood structure $\mathcal{N}_h$ for the adaptive choice of neighborhood (Algorithm 2, line 5), we computed the success rates of each local search operator with the aim of verifying the usefulness the shaking and local search operators of our VNS. The results are presented in Figs. 5 and 6 in the form of box plots.

A successful iteration is defined as every iteration in which a shaking followed by both local searches yields to an improved solution with respect to the current best solution at that time (x_{best}) (lines 8 to 11 of Algorithm 2).

The results of Fig. 5 show that the adaptive choice of neighborhoods based on their past performances conducts a diversified search, and all shaking neighborhoods contribute to improved solutions. It should be noted that these results are separated for small (Fig. 5.A) and large (Fig. 5.B) instances, as we observed that while the number of successful iterations for shaking neighborhoods $\mathcal{N}_3$ and $\mathcal{N}_4$ increases for the large instances, the same does not happen with shaking neighborhoods $\mathcal{N}_1$ and $\mathcal{N}_2$. This may suggest that simply removing all items of one or two categories from all bins may become less effective as the resulting number of required bins increases, while shaking neighborhoods $\mathcal{N}_3$ and $\mathcal{N}_4$ seem to be more effective in terms of avoiding getting trapped at a local optimum.

With respect to the performance of the local searches (Fig. 6), we realized that it was not necessary to separate the results for the small and large instances, as we did not observe significant changes in terms of the number of successful iterations. As expected, local search $\mathcal{L}_2$ yields more successful iterations as it comprises swaps between two items that are assigned to different bins, while $\mathcal{L}_1$ tries to move individual items between bins. Since our fitness function $h(x)$ leads to fuller bin as the VNS heuristic evolves, it may become more difficult for $\mathcal{L}_1$ to find feasible moves in comparison to swaps provided by $\mathcal{L}_2$.

However, both local searches have shown to be effective to improve solutions.

6. Concluding remarks

In this paper, we have addressed the Bin Packing Problem with Compatible Categories (BPCC), a new variant of the BPP that arises in the context of last mile distribution to nanostores. In the BPCC, items are classified into categories, which may or may not be compatible with one another, as opposed to the item-by-item incompatibility addressed in previous literature. Moreover, the typical

Table 11
VNS results for all instances of the evenly distributed 8-category subset.

8-Category Instance set			FFCD			VNS (ran on PC)				
Type	# Items	Cap. Fact (%)	# Opt.[a]	Avg. Gap (%)	Total # Bins	# Opt.[a]	Avg. Gap (%)	Total # Bins	Mean time (s)	Std. Dev. time (s)
AI	202	100	2	1.455	3300	2	1.455	3300	0.380	0.135
		120	23	1.129	2800	42	0.319	2777	0.241	0.196
		150	50	0	2200	50	0	2200	*N/A*	*N/A*
		200	50	0	1650	50	0	1650	*N/A*	*N/A*
	403	100	0	0.782	6652	0	0.752	6650	4.893	2.452
		120	18	0.614	5683	32	0.334	5667	3.648	1.941
		150	0	1.144	4456	1	1.101	4454	3.758	1.507
		200	0	1.493	3350	0	1.493	3350	2.320	1.151
	601	100	0	0.532	9953	0	0.503	9950	22.767	7.442
		120	20	0.380	8650	27	0.277	8641	16.788	7.018
		150	0	0.796	6654	1	0.737	6650	12.075	6.048
		200	0	1.000	5000	0	1.000	5000	12.205	6.474
	802	100	0	0.495	13,316	0	0.376	13,300	55.782	22.398
		120	24	0.258	11,642	34	0.146	11,629	42.861	18.437
		150	4	0.538	8926	45	0.078	8885	8.592	15.751
		200	50	0	6650	50	0	6650	*N/A*	*N/A*
	1003	100	0	0.497	16,683	0	0.300	16,650	131.234	38.555
		120	31	0.136	14,737	37	0.089	14,730	104.711	37.935
		150	23	0.254	11,318	35	0.140	11,305	71.701	38.807
		200	0	0.599	8350	0	0.599	8350	72.383	35.053
ANI	201	100	6	1.333	3300	6	1.333	3300	0.378	0.208
		120	23	1.129	2800	39	0.425	2780	0.209	0.165
		150	50	0	2200	50	0	2200	*N/A*	*N/A*
		200	50	0	1650	50	0	1650	*N/A*	*N/A*
	402	100	0	0.782	6652	0	0.752	6650	4.323	1.617
		120	18	0.614	5683	33	0.299	5665	3.771	1.731
		150	0	1.144	4456	1	1.101	4454	3.876	1.964
		200	0	1.493	3350	0	1.493	3350	2.248	0.988
	600	100	0	0.522	9952	0	0.503	9950	23.262	8.431
		120	20	0.380	8650	28	0.265	8640	17.259	7.355
		150	0	0.796	6654	1	0.737	6650	13.167	6.492
		200	0	1.000	5000	0	1.000	5000	12.303	5.458
	801	100	0	0.503	13,317	0	0.376	13,300	57.538	19.297
		120	24	0.258	11,642	34	0.146	11,629	43.514	18.796
		150	4	0.538	8926	45	0.078	8885	8.198	16.148
		200	50	0	6650	50	0	6650	*N/A*	*N/A*
	1002	100	0	0.497	16,683	0	0.300	16,650	123.612	39.341
		120	31	0.136	14,737	38	0.082	14,729	108.596	39.062
		150	23	0.254	11,318	35	0.140	11,305	65.460	34.704
		200	0	0.599	8350	0	0.599	8350	79.558	38.448

[a] Either L_3 or L_{CPLEX} (when available) lower bounds were used.

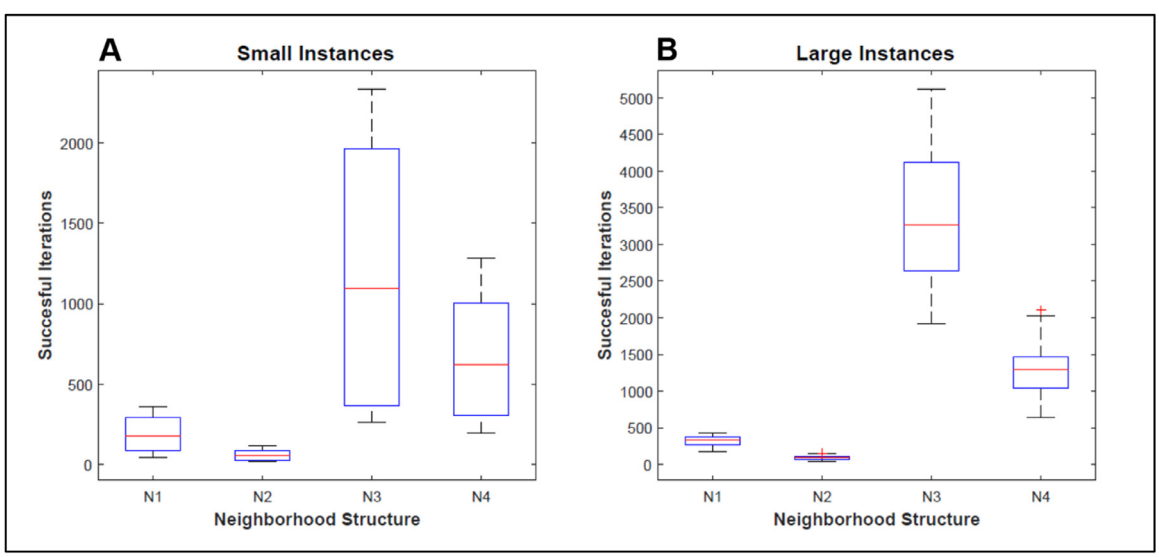

Fig. 5. Performance of the shaking operators.

Table 12
CPLEX and VNS results for the small instances of the two unevenly distributed 6-category subsets.

Instance set				FFCD			CPLEX 12.6 (ran on HPC)					VNS (ran on PC)				
Uneven Class[a]	Type	# Items	Cap. Fact (%)	# Opt.[b]	Avg. Gap (%)	Total # Bins	# Opt. [b]	Avg. Gap (%)	Total # Bins	Mean time[d] (s)	Std. Dev. time[d] (s)	# Opt.[c]	Avg. Gap (%)	Total # Bins	Mean time (s)	Std. Dev. time (s)
U2	AI	202	100	1	1.364	660	1	1.364	660	443.890	*N/A*	1	1.364	660	0.294	0.093
			120	5	0.890	558	9	0.179	553	58.222	44.187	10	0	553	0.079	0.079
			150	10	0	440	10	0	440	4.888	2.400	10	0	440	*N/A*	*N/A*
			200	10	0	330	10	0	330	3.915	0.955	10	0	330	*N/A*	*N/A*
		403	100	0	0.752	1197	0	0.752	1197	*N/A*	*N/A*	0	0.752	1197	2.692	1.432
			120	6	0.349	1143	4	0.780	1141	658.775	753.839	8	0.174	1141	2.004	1.355
			150	0	1.118	895	1	1.011	894	6320.470	*N/A*	1	1.011	894	2.299	1.244
			200	0	1.493	670	0	1.493	670	*N/A*	*N/A*	0	1.493	670	1.659	1.100
	ANI	201	100	0	1.515	660	0	1.515	660	*N/A*	*N/A*	0	1.515	660	0.209	0.082
			120	5	0.890	558	9	0.179	553	89.437	149.039	10	0	553	0.103	0.105
			150	10	0	440	10	0	440	5.399	2.426	10	0	440	*N/A*	*N/A*
			200	10	0	330	10	0	330	3.478	1.057	10	0	330	*N/A*	*N/A*
		402	100	0	0.752	1330	0	0.752	1330	*N/A*	*N/A*	0	0.752	1330	4.077	1.632
			120	6	0.349	1143	4	0.780	1141	896.250	1026.960	8	0.174	1141	2.539	1.731
			150	0	1.118	895	1	1.011	894	1228.420	*N/A*	1	1.011	894	2.485	0.962
			200	0	1.493	670	0	1.493	670	*N/A*	*N/A*	0	1.493	670	1.291	0.565
U4	AI	202	100	1	1.364	660	1	1.364	660	9676.570	*N/A*	1	1.364	660	0.272	0.098
			120	7	0.529	557	9	0.179	554	126.643	249.226	10	0	554	0.186	0.107
			150	10	0	440	10	0	440	5.545	2.208	10	0	440	*N/A*	*N/A*
			200	10	0	330	10	0	330	3.862	1.001	10	0	330	*N/A*	*N/A*
		403	100[e]	0	0.752	1197	0	0.752	1197	*N/A*	*N/A*	0	0.752	1197	3.639	1.775
			120	7	0.349	1143	5	0.865	1142	1054.094	1571.181	8	0.259	1142	2.279	0.768
			150	0	1.118	895	1	1.011	894	4895.330	N/A	1	1.011	894	3.300	1.217
			200	0	1.493	670	0	1.493	670	*N/A*	*N/A*	0	1.493	670	1.523	1.004
	ANI	201	100	0	1.515	660	0	1.515	660	*N/A*	*N/A*	0	1.515	660	0.330	0.205
			120	7	0.529	557	10	0	554	432.497	783.240	10	0	554	0.249	0.150
			150	10	0	440	10	0	440	5.144	1.705	10	0	440	*N/A*	*N/A*
			200	10	0	330	10	0	330	3.559	0.856	10	0	330	*N/A*	*N/A*
		402	100	0	0.752	1330	0	0.752	1330	*N/A*	*N/A*	0	0.752	1330	2.843	1.420
			120	7	0.349	1143	5	0.865	1142	867.038	1176.546	8	0.259	1142	2.756	1.599
			150	0	1.118	895	1	1.011	894	5525.250	*N/A*	1	1.011	894	2.949	1.479
			200	0	1.493	670	0	1.493	670	*N/A*	*N/A*	0	1.493	670	2.137	0.897

[a] U2 and U4 denote subsets with two and four large categories, respectively
[b] Best integer lower bounds obtained by CPLEX after a 5 h time limit (L_{CPLEX}) were used.
[c] Either L_3 or L_{CPLEX} lower bounds were used.
[d] Mean and standard deviation times for CPLEX are given only when optimal solutions could be reached.
[e] The results refer to only 9 instances, since for one single instance (out of the 10 tested for each set) no integer solution could be found by CPLEX running in the HPC after a 5 h time limit.

Table 13
VNS results for all instances of the U2 unevenly 6-category subset.

Uneven-2 Instance set			FFCD			VNS (ran on PC)				
Type	# Items	Cap. Fact (%)	# Opt.[a]	Avg. Gap (%)	Total # Bins	# Opt.[a]	Avg. Gap (%)	Total # Bins	Mean time (s)	Std. Dev. time (s)
AI	202	100	1	1.485	3300	1	1.485	3300	0.294	0.125
		120	32	0.672	2786	42	0.283	2775	0.190	0.123
		150	50	0	2200	50	0	2200	*N/A*	*N/A*
		200	50	0	1650	50	0	1650	*N/A*	*N/A*
	403	100	0	0.752	6650	0	0.752	6650	4.093	2.052
		120	27	0.422	5672	40	0.176	5658	2.532	1.565
		150	0	1.144	4456	1	1.101	4454	2.896	1.604
		200	0	1.493	3350	0	1.493	3350	1.683	0.787
	601	100	0	0.503	9950	0	0.503	9950	17.745	6.147
		120	23	0.313	8644	36	0.163	8631	12.662	5.129
		150	0	0.766	6652	0	0.752	6651	9.909	4.865
		200	0	1.000	5000	0	1.000	5000	9.253	4.256
	802	100	0	0.398	13,303	0	0.376	13,300	40.353	16.072
		120	26	0.215	11,637	34	0.139	11,628	37.097	15.124
		150	20	0.336	8908	47	0.033	8881	7.877	13.072
		200	50	0	6650	50	0	6650	*N/A*	*N/A*
	1003	100	0	0.372	16,662	0	0.300	16,650	98.681	30.638
		120	33	0.116	14,734	40	0.068	14,727	86.648	27.385
		150	24	0.237	11,316	40	0.087	11,299	47.763	30.519
		200	0	0.599	8350	0	0.599	8350	57.933	24.407
ANI	201	100	0	1.515	3300	0	1.515	3300	0.305	0.132
		120	32	0.672	2786	42	0.283	2775	0.178	0.155
		150	50	0	2200	50	0	2200	*N/A*	*N/A*
		200	50	0	1650	50	0	1650	*N/A*	*N/A*
	402	100	0	0.752	6650	0	0.752	6650	3.559	1.764
		120	27	0.422	5672	38	0.211	5660	2.563	1.437
		150	0	1.144	4456	1	1.101	4454	2.862	1.178
		200	0	1.493	3350	0	1.493	3350	1.840	0.881
	600	100	0	0.503	9950	0	0.503	9950	16.890	6.233
		120	23	0.313	8644	36	0.163	8631	12.424	5.448
		150	0	0.766	6652	0	0.752	6651	9.751	4.748
		200	0	1.000	5000	0	1.000	5000	8.990	3.825
	801	100	0	0.391	13,302	0	0.376	13,300	38.353	16.078
		120	26	0.215	11,637	35	0.130	11,627	33.611	15.381
		150	19	0.347	8909	46	0.045	8882	6.985	10.982
		200	50	0	6650	50	0.000	6650	*N/A*	*N/A*
	1002	100	0	0.378	16,663	0	0.300	16,650	99.290	30.477
		120	33	0.116	14,734	40	0.068	14,727	87.133	29.965
		150	25	0.228	11,315	41	0.078	11,298	47.390	26.078
		200	0	0.599	8350	0	0.599	8350	59.806	26.585

[a] Either L_3 or L_{CPLEX} (when available) lower bounds were used.

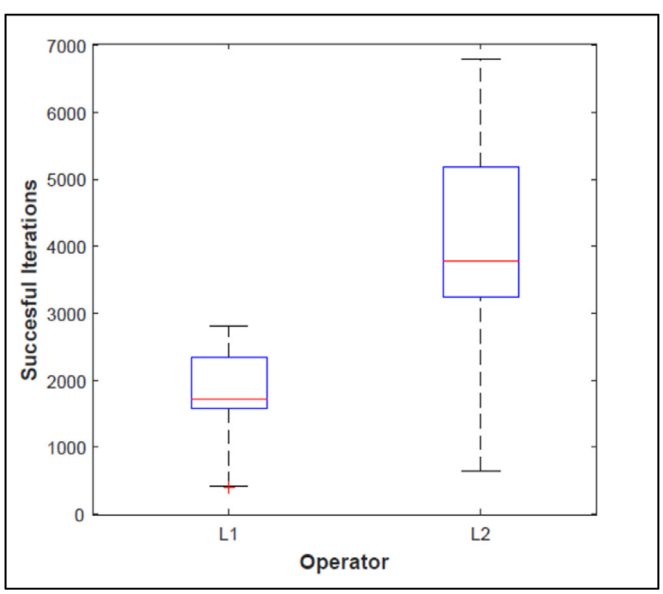

Fig. 6. Performance of the local searches.

characteristics of these distribution systems are such that the distances of line-haul legs of delivery routes are significantly greater than the total distance while the vehicle is performing deliveries. Therefore, as fixed costs prevail over per-mile variable costs, minimizing the number of required vehicles is the key goal to be achieved.

A Variable Neighborhood Search (VNS) algorithm was proposed and tested against large distinct sets of instances derived from two difficult sets of bin packing benchmark problems. Each set comprises 2000 instances that differ in terms of number of categories and the distribution of items among categories. We have also presented two new mathematical formulations for the problem and compared them to the original BPC formulation that considers disjunction constraints (Gendreau et al., 2004), which was also used to help evaluating the quality of our solutions. The results also show that one of the ILP formulations we have proposed (namely the *Cat* approach) yields to the best results in terms of the greatest number of optimal solutions found within reasonable runtimes.

Our VNS algorithm allowed us to reach optimal solutions in very short CPU times using an ordinary PC for the smaller instances, whose optimal results could only be obtained by CPLEX running on a very powerful high-performance computer. The heuristic also yielded to high-quality results in very short running times for the larger instances of the different subsets, for which

Table 14
VNS results for all instances of the U4 unevenly 6-category subset.

Uneven-4 Instance set			FFCD			VNS (ran on PC)				
Type	# Items	Cap. Fact (%)	# Opt.[a]	Avg. Gap (%)	Total # Bins	# Opt.[a]	Avg. Gap (%)	Total # Bins	Mean time (s)	Std. Dev. time (s)
AI	202	100	1	1.485	3300	1	1.485	3300	0.260	0.113
		120	33	0.672	2787	42	0.283	2776	0.193	0.155
		150	50	0	2200	50	0	2200	*N/A*	*N/A*
		200	50	0	1650	50	0	1650	*N/A*	*N/A*
	403	100	0	0.752	6650	0	0.752	6650	3.863	1.746
		120	27	0.455	5674	39	0.210	5660	2.630	1.166
		150	0	1.122	4455	1	1.101	4454	2.836	1.250
		200	0	1.493	3350	0	1.493	3350	1.536	0.755
	601	100	0	0.503	9950	0	0.503	9950	18.494	5.614
		120	25	0.311	8644	33	0.196	8634	12.556	6.161
		150	0	0.766	6652	0	0.766	6652	10.198	4.149
		200	0	1.000	5000	0	1.000	5000	8.423	3.661
	802	100	0	0.428	13,307	0	0.376	13,300	40.124	16.524
		120	31	0.172	11,632	37	0.113	11,625	36.281	14.308
		150	14	0.414	8915	45	0.055	8883	6.871	11.668
		200	50	0	6650	50	0	6650	*N/A*	*N/A*
	1003	100	0	0.390	16,665	0	0.300	16,650	101.674	26.257
		120	35	0.102	14,732	40	0.068	14,727	87.335	31.145
		150	33	0.149	11,306	42	0.070	11,297	48.241	26.701
		200	0	0.599	8350	0	0.599	8350	55.616	24.625
ANI	201	100	0	1.515	3300	0	1.515	3300	0.259	0.104
		120	33	0.672	2787	42	0.283	2776	0.158	0.102
		150	50	0	2200	50	0	2200	*N/A*	*N/A*
		200	50	0	1650	50	0	1650	*N/A*	*N/A*
	402	100	0	0.752	6650	0	0.752	6650	3.874	1.533
		120	27	0.455	5674	41	0.174	5658	2.440	1.071
		150	0	1.122	4455	1	1.101	4454	2.842	1.418
		200	0	1.493	3350	0	1.493	3350	1.719	0.806
	600	100	0	0.503	9950	0	0.503	9950	18.369	6.296
		120	25	0.311	8644	33	0.196	8634	12.737	5.850
		150	0	0.766	6652	0	0.766	6652	9.660	4.467
		200	0	1.000	5000	0	1.000	5000	8.613	4.093
	801	100	0	0.428	13,307	0	0.376	13,300	37.158	14.587
		120	31	0.172	11,632	37	0.113	11,625	32.828	13.341
		150	12	0.437	8917	45	0.055	8883	6.925	12.410
		200	50	0	6650	50	0	6650	*N/A*	*N/A*
	1002	100	0	0.408	16,668	0	0.300	16,650	101.053	29.523
		120	35	0.102	14,732	41	0.061	14,726	81.051	31.168
		150	33	0.149	11,306	42	0.070	11,297	45.239	25.880
		200	0	0.599	8350	0	0.599	8350	51.514	21.913

[a] Either L_3 or L_{CPLEX} (when available) lower bounds were used.

not only optimal results could not be obtained, but also not even a single feasible integer solution could be found in most cases.

Moreover, our VNS is simple and easy to implement, and yields to high-quality solutions in very short times for all different instances sets that were tested. In addition, it does not require any complex and extensive parameter calibration or specific instance-based settings to reach good quality solutions. Therefore, it can be applied to solve bin packing problems with compatibility constraints that arise in the context of real-world applications related to last mile distribution to nanostores.

Since, to the best of our knowledge, this is the first study of the BPCC, there may be plenty of possibilities for further investigation, involving both exact and heuristic algorithms. Exact algorithms should try to focus on deriving efficient lower bounds based on the set covering formulation and the Branch-and-Price algorithm originally proposed by Muritiba et al. (2010) to the BPC. Other related problems such as the generalization of the BPCC for problems with higher dimensions, such as the BPC-2D (Muritiba et al., 2010) or the 2-DVPP (Buljubašić & Vasquez, 2016) could also be addressed. We leave them all as topics for future research.

Acknowledgments

The authors acknowledge the use of computational facilities of LCCA-Laboratory of Advanced Scientific Computation of the University of São Paulo (LCCA-USP). The first and the last authors also acknowledges CNPq (Brazil's National Council for Scientific and Technological Development) financial support [grant numbers 830616/1999-3 and 304069/2010-8, respectively]; and the third author acknowledges CAPES (Brazil's Coordination for the Improvement of Higher Education Personnel) financial support [grant number 1571341].

References

Bortfeldt, A., & Homberger, J. (2013). Packing first, routing second-a heuristic for the vehicle routing and loading problem. *Computers & Operations Research, 40*, 873–885. doi:10.1016/j.cor.2012.09.005.

Buljubašić, M., & Vasquez, M. (2016). Consistent neighborhood search for one-dimensional bin packing and two-dimensional vector packing. *Computers & Operations Research, 76*, 12–21. doi:10.1016/j.cor.2016.06.009.

Caprara, A., Dell'Amico, M., Díaz-Díaz, J. C., Iori, M., & Rizzi, R. (2015). Friendly bin packing instances without integer round-up property. *Mathematical Programming, 150*, 5–17. doi:10.1007/s10107-014-0791-z.

Coffman, E. G., Csirik, J., Galambos, G., Martello, S., & Vigo, D. (2013). Bin packing approximation algorithms: Survey and classification. In P. M. Pardalos, D.-Z. Du, & R. L. Graham (Eds.), *Handbook of combinatorial optimization* (pp. 455–531). New York, NY: Springer. doi:10.1007/978-1-4419-7997-1_35.

Delorme, M., Iori, M., & Martello, S. (2016). Bin packing and cutting stock problems: Mathematical models and exact algorithms. *European Journal of Operational Research, 255*, 1–20. doi:10.1016/j.ejor.2016.04.030.

Dowsland, K. A., & Dowsland, W. B. (1992). Packing problems. *European Journal of Operational Research, 56*, 2–14. doi:10.1016/0377-2217(92)90288-K.

Dusberger, F., & Raidl, G. R. (2015). Solving the 3-staged 2-dimensional cutting stock problem by dynamic programming and variable neighborhood search. *Electronic Notes in Discrete Mathematics, 47*, 133–140. doi:10.1016/j.endm.2014.11.018.

Fleszar, K., & Hindi, K. S. (2002). New heuristics for one-dimensional bin-packing. *Computers & Operations Research, 29*, 821–839. doi:10.1016/S0305-0548(00)00082-4.

Fransoo, J. C., Blanco, E. E., & Mejia-Argueta, C. (2017). *Reaching 50 million nanostores: Retail distribution in emerging megacities* (1st ed). CreateSpace Independent Publishing Platform.

Fuellerer, G., Doerner, K. F., Hartl, R. F., & Iori, M. (2010). Metaheuristics for vehicle routing problems with three-dimensional loading constraints. *European Journal of Operational Research, 201*, 751–759. doi:10.1016/j.ejor.2009.03.046.

Gámez Albán, H. M., Soto Cardona, O. C., Mejía Argueta, C., & Sarmiento, A. T. (2015). A cost-efficient method to optimize package size in emerging markets. *European Journal of Operational Research, 241*, 917–926. doi:10.1016/j.ejor.2014.09.020.

Gendreau, M., Iori, M., Laporte, G., & Martello, S. (2006). A tabu search algorithm for a routing and container loading problem. *Transportation Science, 40*, 342–350. doi:10.1287/trsc.1050.0145.

Gendreau, M., Laporte, G., & Semet, F. (2004). Heuristics and lower bounds for the bin packing problem with conflicts. *Computers & Operations Research, 31*, 347–358. doi:10.1016/S0305-0548(02)00195-8.

Hansen, P., & Mladenović, N. (2001). Variable neighborhood search: Principles and applications. *European Journal of Operational Research, 130*, 449–467. doi:10.1016/S0377-2217(00)00100-4.

Hansen, P., Mladenovic, N., Brimberg, J., & Moreno Pérez, J. A. (2010). Handbook of metaheuristics. *Handbook of metaheuristics*. Boston, MA: Springer. doi:10.1007/978-1-4419-1665-5.

Hemmelmayr, V., Schmid, V., & Blum, C. (2012). Variable neighbourhood search for the variable sized bin packing problem. *Computers & Operations Research, 39*, 1097–1108. doi:10.1016/j.cor.2011.07.003.

Henderson, R. (1995). Job scheduling under the portable batch system. In D. G. Feitelson, & L. Rudolph (Eds.), *Proceedings of the job scheduling strategies for parallel processing: IPPS '95 workshop Santa Barbara, CA, USA, April 25, 1995* (pp. 279–294). Berlin, Heidelberg: Springer. doi:10.1007/3-540-60153-8_34.

Jansen, K. (1999). An approximation scheme for bin packing with conflicts. *Journal of Combinatorial Optimization, 3*, 363–377. doi:10.1023/A:1009871302966.

Jansen, K., & Öhring, S. (1997). Approximation algorithms for time constrained scheduling. *Information and Computation, 132*, 85–108. doi:10.1006/inco.1996.2616.

Khanafer, A., Clautiaux, F., & Talbi, E. (2010). New lower bounds for bin packing problems with conflicts. *European Journal of Operational Research, 206*, 281–288. doi:10.1016/j.ejor.2010.01.037.

Martello, S., & Toth, P. (1990a). *Knapsack problems: Algorithms and computer implementations* (1st ed). Chichester, England: John Wiley & Sons.

Martello, S., & Toth, P. (1990b). Lower bounds and reduction procedures for the bin packing problem. *Discrete Applied Mathematics, 28*, 59–70. doi:10.1016/0166-218X(90)90094-S.

Masson, R., Vidal, T., Michallet, J., Penna, P. H. V., Petrucci, V., Subramanian, A., et al. (2013). An iterated local search heuristic for multi-capacity bin packing and machine reassignment problems. *Expert Systems with Applications, 40*, 5266–5275. doi:10.1016/j.eswa.2013.03.037.

Mladenović, N., & Hansen, P. (1997). Variable neighborhood search. *Computers & Operations Research, 24*, 1097–1100. doi:10.1016/S0305-0548(97)00031-2.

Muritiba, A. E. F., Iori, M., Malaguti, E., & Toth, P. (2010). Algorithms for the bin packing problem with conflicts. *INFORMS Journal on Computing, 22*, 401–415. doi:10.1287/ijoc.1090.0355.

Pereira, J., & Vilà, M. (2015). Variable neighborhood search heuristics for a test assembly design problem. *Expert Systems with Applications, 42*, 4805–4817. doi:10.1016/j.eswa.2015.01.057.

Quiroz-Castellanos, M., Cruz-Reyes, L., Torres-Jimenez, J., Gómez, S. C., Huacuja, H. J. F., & Alvim, A. C. F. (2015). A grouping genetic algorithm with controlled gene transmission for the bin packing problem. *Computers & Operations Research, 55*, 52–64. doi:10.1016/j.cor.2014.10.010.

Respen, J., & Zufferey, N. (2017). Metaheuristics for truck loading in the car production industry. *International Transactions in Operational Research, 24*, 277–301. doi:10.1111/itor.12306.

Ropke, S., & Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science, 40*, 455–472. doi:10.1287/trsc.1050.0135.

Sadykov, R., & Vanderbeck, F. (2013). Bin packing with conflicts: A generic branch-and-price algorithm. *INFORMS Journal on Computing, 25*, 244–255. doi:10.1287/ijoc.1120.0499.

Schrimpf, G., Schneider, J., Stamm-Wilbrandt, H., & Dueck, G. (2000). Record breaking optimization results using the ruin and recreate principle. *Journal of Computational Physics, 159*, 139–171. doi:10.1006/jcph.1999.6413.

Shaw, P. (1998). Using constraint programming and local search methods to solve vehicle routing problems. In M. Maher, & J. F. Puget (Eds.), *Proceedings of the principles and practice of constraint programming — CP98: 1520*. Berlin Heidelberg: Springer Lecture Notes in Computer Science.

Stawowy, A. (2008). Evolutionary based heuristic for bin packing problem. *Computers & Industrial Engineering, 55*, 465–474. doi:10.1016/j.cie.2008.01.007.

Zhu, W., Qin, H., Lim, A., & Wang, L. (2012). A two-stage tabu search algorithm with enhanced packing heuristics for the 3L-CVRP and M3L-CVRP. *Computers & Operations Research, 39*, 2178–2195. doi:10.1016/j.cor.2011.11.001.